



برنامه نویسی به زبان سی

ساختارها و لیست های پیوندی

دکتر مهران صفایانی

safayani@iut.ac.ir

safayani.iut.ac.ir



<https://www.aparat.com/mehran.safayani>



https://github.com/safayani/Programming_Basics_course



Structures in C

- **Definition:** A **structure** is a derived data type constructed from objects of other types.
- **Key Concept:** Structures allow grouping of related data of **different types** under one name.
- **Keyword:** struct

Defining a Structure

```
struct employee {  
    char firstName[20];  
    char lastName[20];  
    int age;  
    double hourlySalary;  
};
```

- **Structure Tag:** card (used to declare variables of this type, e.g., struct card)
- **Members:** Variables declared inside the braces (face, suit). Must have unique names within the struct.
- **Termination:** Structure definition ends with a semicolon.

Self-Referential Structures

- A structure **cannot** contain a variable of its own type.
- A structure **can** contain a **pointer** to its own type.
- Used to build linked data structures (e.g., linked lists, trees).
- **Example:**

```
struct Node {  
    int data;  
    struct Node next; // ERROR: Invalid!  
};  
  
struct employee {  
    char firstName[20];  
    // ... other members  
    struct employee *managerPtr; // Pointer to another employee struct  
};
```

Defining Structure Variables

- A structure definition is a **blueprint**; it does not allocate memory.
- **Three ways to define variables:**

1. **Separate declaration:**

```
struct employee myEmployee;  
struct employee EM[52];  
struct employee *employeePtr;
```

2. **Definition with variable list:**

```
struct employee {  
    // members  
} myEmployee, EM[52], * employeePtr;
```

3. **Without a tag name**

(must define variables immediately)

```
// NO TAG NAME - must declare variables NOW  
struct {      // ← No "employee" tag!  
    // members  
} myEmployee, EM[52], * employeePtr;
```

Operations Allowed on Structures

- **Assignment:** Assign one struct to another of the **same type** (copies all members; for pointers, copies the address, not the pointed-to data).
- **Address:** Take the address of a struct using &.
- **Member Access:** Access members using the dot (.) or arrow (->) operators.
- **sizeof:** Use sizeof to get the total size in bytes.
- **Comparison:** Structures **cannot** be compared using == or !=.

Initializing Structures with Initializer Lists

- Structures can be initialized using initializer lists (similar to arrays)
- **Syntax:**

```
struct card myCard = {"Three", "Hearts"};  
    myCard.face = "Three"  
    myCard.suit = "Hearts"
```

Values are assigned to members in the order declared in structure definition

Initialization Rules

- **Partial Initialization:**

- Fewer initializers than members → remaining members set to **0** or **NULL**

```
struct example {
```

```
    int id;
```

```
    double value;
```

```
    char *name;
```

```
};
```

```
struct example partial = {10}; // id=10, value=0.0, name=NULL
```

- **Assignment:**

- Can assign entire structures of same type: struct2 = struct1;
- Can assign to individual members: myCard.face = "Ace";

Accessing Structure Members - Dot Operator (.)

- **Structure Member Operator** (dot operator .)
- Used to access members via structure **variable name**
- **Example:**

```
struct employee myEmployee = {"Ali", "Ahmadi", 20, 300.0};
```

```
printf("%s", myEmployee.firstName); // displays "Ali"
```

Usage: structure_variable.member_name

```
struct employee {  
    char firstName[20];  
    char lastName[20];  
    int age;  
    double hourlySalary;  
};
```

Accessing Structure Members - Arrow Operator (->)

- **Structure Pointer Operator** (arrow operator ->)
- Used to access members via **pointer** to structure
- **Example:**

```
struct employee {  
    char firstName[20];  
    char lastName[20];  
    int age;  
    double hourlySalary;  
};
```

```
struct employee myEmployee = {"Ali", "Ahmadi", 20, 300.0};
```

```
struct card *employeePtr = &myEmployee;
```

```
printf("%s", employeePtr->firstName); // displays "Ali"
```

Equivalent to: (*employeePtr).firstName (parentheses required)

Operator Precedence & Spacing

- **Precedence:**

- `.` and `->` have highest precedence (group left-to-right)

- `(*ptr).member` needs parentheses because `.` has higher precedence than `*`

- **Spacing Conventions:**

- **DO:** `ptr->member`, `var.member` (no spaces around operators)

- **DON'T:** `ptr -> member`, `var . member` (syntax error for `->` with spaces)

```

#include <stdio.h>
struct student {
    const char *name; // Student name
    int id;           // Student ID
    float gpa;        // Grade point average
};
int main(void) {
    struct student stu = {"Alice", 12345, 3.8};
    struct student *stuPtr = &stu;

    printf("Student Info:\n");
    printf("%s, ID: %d, GPA: %.2f\n", stu.name, stu.id, stu.gpa);
    printf("%s, ID: %d, GPA: %.2f\n", stuPtr->name, stuPtr->id, stuPtr->gpa);
    printf("%s, ID: %d, GPA: %.2f\n", (*stuPtr).name, (*stuPtr).id, (*stuPtr).gpa);
}

```

```

Student Info:
Alice, ID: 12345, GPA: 3.80
Alice, ID: 12345, GPA: 3.80
Alice, ID: 12345, GPA: 3.80

```

Using Structures with Functions

- **Three Ways to Pass Structures to Functions:**
 - **Individual members** (passed by value)
 - **Entire structure object** (passed by value – entire struct copied)
 - **Pointer to structure object** (passed by reference)

Individual members (passed by value)

```
void update_data(int id, float score) {  
    id += 100;    // Only local copy changes  
    score *= 1.1; // Only local copy changes  
}
```

// Usage:

```
struct Student s = {101, 85.5};  
update_data(s.id, s.score); // s remains unchanged
```

Pass Entire Structure (by value - copy)

```
struct Point modify_point(struct Point p) {  
    p.x += 10; // Only local copy changes  
    p.y += 20; // Only local copy changes  
    return p; // Return modified copy  
}
```

// Usage:

```
struct Point p1 = {5, 10};  
struct Point p2 = modify_point(p1); // p1 unchanged, p2 modified
```

Pass Pointer to Structure (by reference)

```
void update_student(struct Student *s) {  
    s->id += 100; // Original changes  
    s->score += 5; // Original changes  
}
```

// Usage:

```
struct Student stu = {101, 85.5};  
update_student(&stu); // stu IS modified
```

typedef – Creating Type Aliases

- **Purpose:** Create synonyms/aliases for existing types
- **Common use:** Simplify struct declarations
- **Basic syntax:**

```
typedef existing_type new_type_name;
```

- **Example for structures:**

```
typedef struct employee Employee; // Employee is now alias for struct employee
```

```
Employee EM[52];
```

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define EMPLOYEES 52
#define DEPARTMENTS 13
struct employee {
    const char *name;    // define pointer name
    const char *dept;    // define pointer department
};
typedef struct employee Employee; // new type name for struct employee
void fillEmployees(Employee * const employees, const char *names[], const char *depts[]);
void shuffleEmployees(Employee * const employees);
void displayEmployees(const Employee * const employees);
```

```
int main(void) {  
    Employee employees[EMPLOYEES]; // define array of Employees  
    const char *names[] = {  
        "John", "Sarah", "Mike", "Emma", "David",  
        "Lisa", "James", "Anna", "Robert", "Maria",  
        "William", "Susan", "Charles"  
    };  
    const char *depts[] = {  
        "Sales", "IT", "HR", "Finance"  
    };  
  
    srand(time(NULL)); // randomize  
  
    fillEmployees(employees, names, depts); // load the employees  
    shuffleEmployees(employees);           // put Employees in random order  
    displayEmployees(employees);           // display all 52 Employees  
    return 0;}  

```

```
void fillEmployees(Employee * const employees, const char *names[],
                  const char *depts[]) {
    for (size_t i = 0; i < EMPLOYEES; ++i) {
        employees[i].name = names[i % DEPARTMENTS];
        employees[i].dept = depts[i / DEPARTMENTS];    }}
```

```
void shuffleEmployees(Employee * const employees) {
    for (size_t i = 0; i < EMPLOYEES; ++i) {
        size_t j = rand() % EMPLOYEES;
        Employee temp = employees[i];
        employees[i] = employees[j];
        employees[j] = temp;    }}
```

Susan	in	HR	Charles	in	IT	James	in	HR	Emma	in	IT
Charles	in	HR	Lisa	in	Finance	Charles	in	Sales	David	in	IT
Lisa	in	HR	Maria	in	HR	Mike	in	Finance	John	in	IT
Maria	in	IT	Robert	in	HR	William	in	Sales	Emma	in	Sales
John	in	Sales	Maria	in	Finance	Mike	in	Sales	William	in	HR
Emma	in	HR	James	in	IT	Anna	in	Sales	David	in	Sales
Susan	in	Finance	Robert	in	Sales	Mike	in	HR	Mike	in	IT
Anna	in	HR	Anna	in	Finance	Charles	in	Finance	Sarah	in	IT
Lisa	in	Sales	David	in	HR	Emma	in	Finance	David	in	Finance
Sarah	in	Finance	Maria	in	Sales	Susan	in	Sales	William	in	Finance
William	in	IT	Susan	in	IT	Robert	in	Finance	Sarah	in	HR
John	in	Finance	James	in	Finance	Lisa	in	IT	John	in	HR
Anna	in	IT	Sarah	in	Sales	James	in	Sales	Robert	in	IT

```
void displayEmployees(const Employee * const employees) {
    for (size_t i = 0; i < EMPLOYEES; ++i) {
        printf("%-8s in %-10s%s", employees[i].name, employees[i].dept,
              (i + 1) % 4 ? " " : "\n");    }}
```

Limitations of Fixed-Size Structures

- **Current Knowledge:**
 - ✓ One-dimensional arrays
 - ✓ Two-dimensional arrays
 - ✓ Structures (structs)
- **Key Limitation:**
 - Fixed size at compile time
 - Cannot grow/shrink at runtime
 - Memory may be wasted or insufficient
- **Need:**
 - Data structures that adapt during execution
 - Efficient memory utilization
 - Flexible data organization

Dynamic Memory Management

- **Two Essential Components:**
 - **Allocation:** Obtain memory at runtime (malloc)
 - **Deallocation:** Release unused memory (free)
- **Why Dynamic?**
 - Fixed-size arrays waste memory
 - Unknown data size at compile time
 - Need structures that grow/shrink at runtime
- **Key Tools:**
 - malloc() - Memory allocation
 - free() - Memory deallocation
 - sizeof - Determine size needed

The malloc Function

- **Purpose:** Request memory at execution time
- **Syntax:**

```
void *malloc(size_t size);
```

- **Usage with Structures:**

```
struct node *newPtr;
```

```
newPtr = malloc(sizeof(struct node));
```

- **What Happens:**
 - Calculates struct node size
 - Requests that many bytes from OS
 - Returns void* pointer to allocated memory

Critical malloc Safety Practices

- **ALWAYS Check for NULL:**

```
struct node *newPtr = malloc(sizeof(struct node));  
if (newPtr == NULL) {  
    printf("Memory allocation failed!\n");  
    exit(1); // Exit gracefully  
}
```

- **Why Check?**

- malloc returns NULL if no memory available
- Accessing NULL pointer → **Program crash**

The free Function

- **Purpose:** Return memory to system
- **Syntax:**

```
void free(void *ptr);
```

- **Usage:**

```
free(newPtr);    // Return memory
```

```
newPtr = NULL;    // ★ CRITICAL: Set pointer to NULL
```

- **Why Set to NULL?**
 - Prevents "dangling pointer" errors
 - Prevents accidental reuse of freed memory
 - Makes debugging easier

Common Memory Management Errors

- **1. Memory Leaks**

```
void leak_example() {  
    struct node *ptr = malloc(sizeof(struct node));  
    // Forgot to call free(ptr) before function ends!  
}
```

- **Result:** Memory never returned → System runs out

- **2. Dangling Pointers**

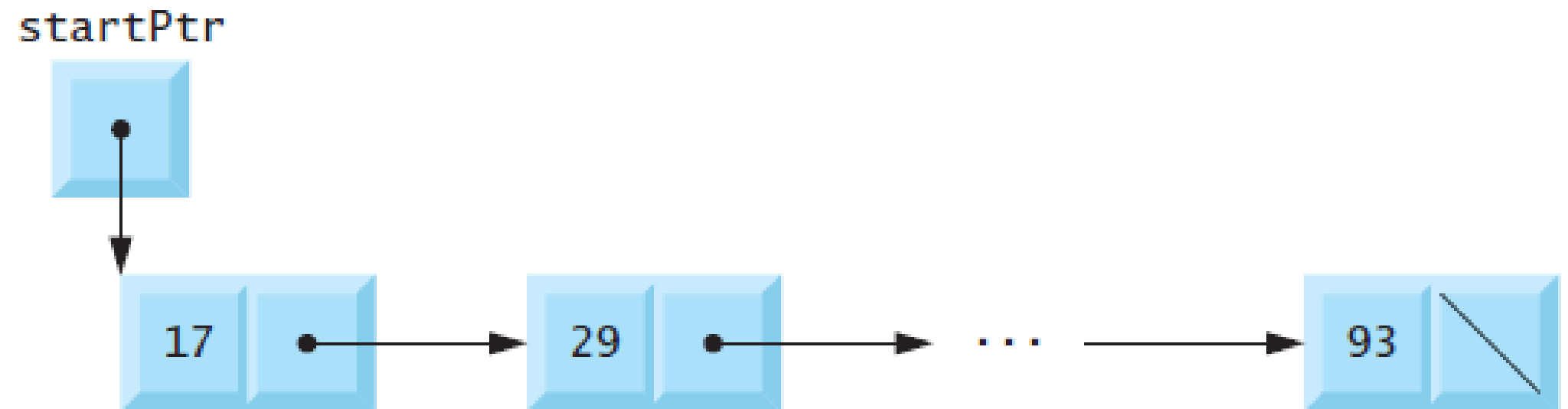
```
struct node *ptr = malloc(sizeof(struct node));  
free(ptr);  
ptr->data = 10; // ✗ CRASH: Accessing freed memory
```

- **3. Double Free**

```
free(ptr);  
free(ptr); // ✗ CRASH: Freeing same memory twice
```

What is a Linked List?

- **Definition:**
A linear collection of self-referential nodes connected by pointers



- **Key Characteristics:**
 - Access via pointer to first node
 - Dynamic size (grows/shrinks at runtime)
 - Non-contiguous memory allocation
 - Sequential access (not random access)

Linked List Node Structure

- **Self-Referential Structure:**

```
struct ListNode {  
    char data;           // Data storage  
    struct ListNode *next; // Pointer to next node  
};  
typedef struct ListNode ListNode;  
typedef ListNode *ListNodePtr;
```

- **Why Self-Referential?**

- Each node points to another node of same type
- Creates chainable structure
- Enables dynamic linking

Linked Lists vs. Arrays

Aspect	Arrays	Linked Lists
Size	Fixed at compile time	Dynamic (grows/shrinks)
Memory	Contiguous allocation	Non-contiguous allocation
Access	Random	Sequential
Insertion	Slow (shifting needed)	Fast
Memory Use	May waste space	Efficient but overhead
Implementation	Simple syntax	More complex pointers

When to Use Linked Lists

- **Use Linked Lists When:**

- Number of elements unpredictable
- Frequent insertions/deletions needed
- Memory efficiency important
- Data needs to stay sorted during insertions

- **Use Arrays When:**

- Fixed/known number of elements
- Random access required
- Memory locality important (cache performance)
- Simple implementation needed

Linked List Operations - Insertion

- **Three Insertion Cases:**
 - Insert at beginning (easiest)
 - Insert at end
 - Insert in middle

- **Insert at Beginning:**

```
typedef struct listNode ListNode;    // ListNode is a struct
typedef ListNode *ListNodePtr;      // ListNodePtr is a pointer to ListNode
void insertAtFront(ListNodePtr *head, char value) {
    ListNodePtr newNode = malloc(sizeof(ListNode));
    newNode->data = value;
    newNode->next = *head; // Point to old first node
    *head = newNode;      // Update head pointer
}
```

```

1 // fig12_01.c
2 // Inserting and deleting nodes in a list
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 // self-referential structure
7 struct listNode {
8     char data; // each listNode contains a character
9     struct listNode *nextPtr; // pointer to next node
10 };
11
12 typedef struct listNode ListNode; // synonym for struct listNode
13 typedef ListNode *ListNodePtr; // synonym for ListNode*
14
15 // prototypes
16 void insert(ListNodePtr *sPtr, char value);
17 char delete(ListNodePtr *sPtr, char value);
18 int isEmpty(ListNodePtr sPtr);
19 void printList(ListNodePtr currentPtr);
20 void instructions(void);

```

```

22 int main(void) {
23     ListNodePtr startPtr = NULL; // initially there are no nodes
24     char item = '\0'; // char entered by user
25
26     instructions(); // display the menu
27     printf("%s", "? ");
28     int choice = 0; // user's choice
29     scanf("%d", &choice);
30
31     // loop while user does not choose 3
32     while (choice != 3) {
33         switch (choice) {
34             case 1: // insert an element
35                 printf("%s", "Enter a character: ");
36                 scanf("\n%c", &item);
37                 insert(&startPtr, item); // insert item in list
38                 printList(startPtr);
39                 break;
40             case 2: // delete an element
41                 if (!isEmpty(startPtr)) { // if list is not empty
42                     printf("%s", "Enter character to be deleted: ");
43                     scanf("\n%c", &item);
44
45                     // if character is found, remove it
46                     if (delete(&startPtr, item)) { // remove item
47                         printf("%c deleted.\n", item);
48                         printList(startPtr);
49                     }
50                     else {
51                         printf("%c not found.\n\n", item);
52                     }
53
54                     else {
55                         puts("List is empty.\n");
56                     }
57
58                     break;
59                     default:
60                         puts("Invalid choice.\n");
61                         instructions();
62                         break;
63                 }
64
65                 printf("%s", "? ");
66                 scanf("%d", &choice);
67             } // end while
68
69             puts("End of run.");
70         }
71
72         // display program instructions to user
73         void instructions(void) {
74             puts("Enter your choice:\n"
75                 " 1 to insert an element into the list.\n"
76                 " 2 to delete an element from the list.\n"
77                 " 3 to end.");
78         }
79

```

```
Enter your choice:
    1 to insert an element into the list.
    2 to delete an element from the list.
    3 to end.
? 1
Enter a character: B
The list is:
B --> NULL

? 1
Enter a character: A

The list is:
A --> B --> NULL

? 1
Enter a character: C
The list is:
A --> B --> C --> NULL

? 2
Enter character to be deleted: D
D not found.
```

```
? 2
Enter character to be deleted: B
B deleted.
The list is:
A --> C --> NULL
```

```
? 2
Enter character to be deleted: C
C deleted.
The list is:
A --> NULL
```

```
? 2
Enter character to be deleted: A
A deleted.
List is empty.
```

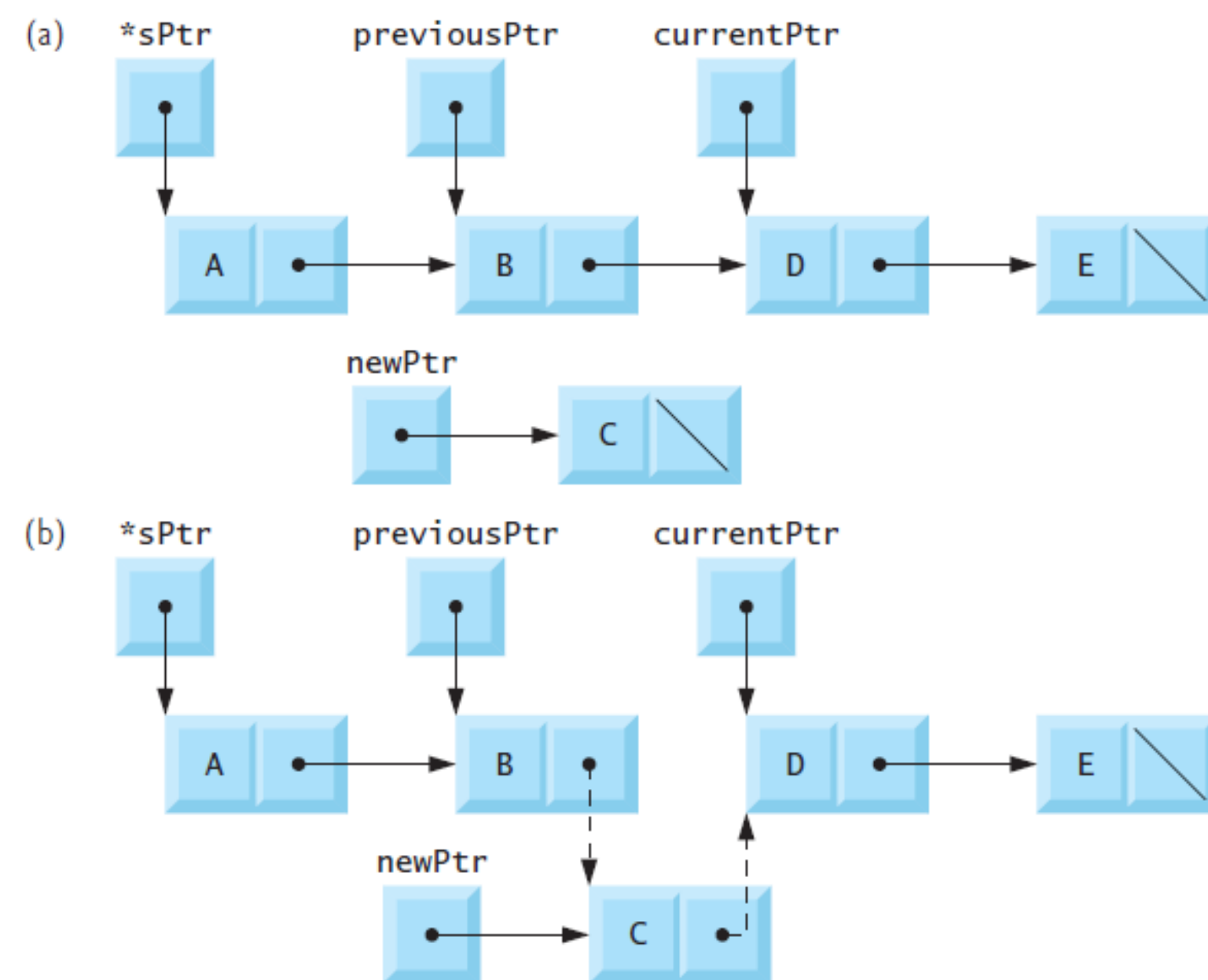
```
? 4
Invalid choice.
```

```
Enter your choice:
    1 to insert an element into the list.
    2 to delete an element from the list.
    3 to end.
? 3
End of run.
```

```

80 // insert a new value into the list in sorted order
81 void insert(ListNodePtr *sPtr, char value) {
82     ListNodePtr newPtr = malloc(sizeof(ListNode)); // create node
83
84     if (newPtr != NULL) { // is space available?
85         newPtr->data = value; // place value in node
86         newPtr->nextPtr = NULL; // node does not link to another node
87
88         ListNodePtr previousPtr = NULL;
89         ListNodePtr currentPtr = *sPtr;
90
91         // loop to find the correct location in the list
92         while (currentPtr != NULL && value > currentPtr->data) {
93             previousPtr = currentPtr; // walk to ...
94             currentPtr = currentPtr->nextPtr; // ... next node
95         }
96
97         // insert new node at beginning of list
98         if (previousPtr == NULL) {
99             newPtr->nextPtr = *sPtr;
100             *sPtr = newPtr;
101         }
102         else { // insert new node between previousPtr and currentPtr
103             previousPtr->nextPtr = newPtr;
104             newPtr->nextPtr = currentPtr;
105         }
106     }
107     else {
108         printf("%c not inserted. No memory available.\n", value);
109     }
110 }
111

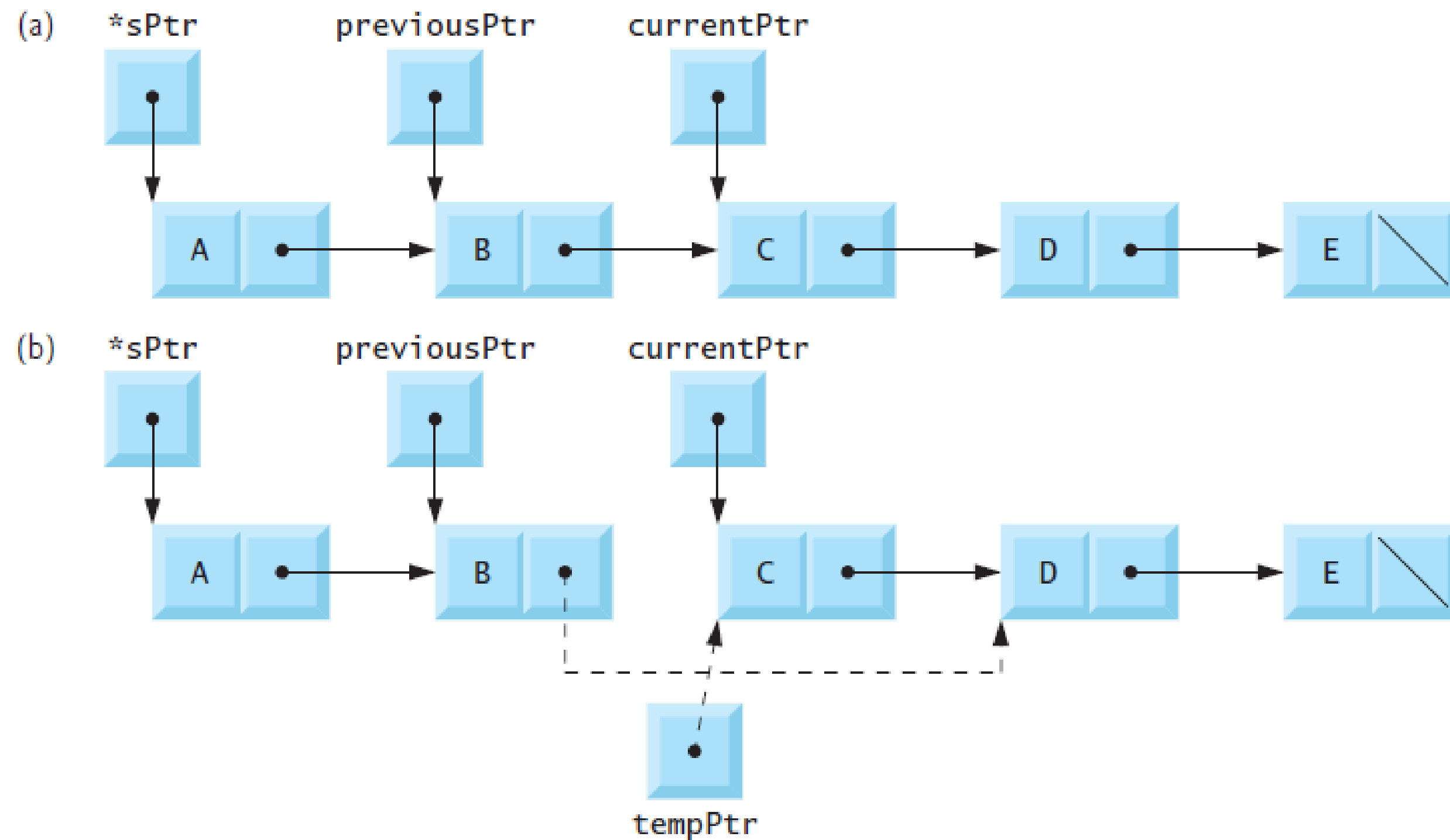
```



```

112 // delete a list element
113 char delete(ListNodePtr *sPtr, char value) {
114     // delete first node if a match is found
115     if (value == (*sPtr)->data) {
116         ListNodePtr tempPtr = *sPtr; // hold onto node being removed
117         *sPtr = (*sPtr)->nextPtr; // de-thread the node
118         free(tempPtr); // free the de-threaded node
119         return value;
120     }
121     else {
122         ListNodePtr previousPtr = *sPtr;
123         ListNodePtr currentPtr = (*sPtr)->nextPtr;
124
125         // loop to find the correct location in the list
126         while (currentPtr != NULL && currentPtr->data != value) {
127             previousPtr = currentPtr; // walk to ...
128             currentPtr = currentPtr->nextPtr; // ... next node
129         }
130
131         // delete node at currentPtr
132         if (currentPtr != NULL) {
133             ListNodePtr tempPtr = currentPtr;
134             previousPtr->nextPtr = currentPtr->nextPtr;
135             free(tempPtr);
136             return value;
137         }
138     }
139
140     return '\0';
141 }
142

```



```

143 // return 1 if the list is empty, 0 otherwise
144 int isEmpty(ListNodePtr sPtr) {
145     return sPtr == NULL;
146 }
147
148 // print the list
149 void printList(ListNodePtr currentPtr) {
150     // if list is empty
151     if (isEmpty(currentPtr)) {
152         puts("List is empty.\n");
153     }
154     else {
155         puts("The list is:");
156
157         // while not the end of the list
158         while (currentPtr != NULL) {
159             printf("%c --> ", currentPtr->data);
160             currentPtr = currentPtr->nextPtr;
161         }
162
163         puts("NULL\n");
164     }
165 }

```